
1 CONFIGURACIONES

Nota: Los términos *problema sobre configuraciones* y otros relacionados que aparecen en estas notas no forman parte de una terminología estándar. Distintos autores pueden emplear nombres diferentes para referirse a los mismos tipos de problemas, o bien estos son agrupados de otra manera. El propósito de la terminología adoptada aquí es únicamente facilitar la organización del material y disponer de un lenguaje común para referirnos a familias de problemas con características similares.

Los problema sobre configuraciones se pueden dividir en dos categorías principales: problemas sobre configuraciones *estáticas* y *dinámicas*.

1.1. Configuraciones estáticas

En este caso, partimos de una estructura fija (normalmente un tablero, una lista o un arreglo) donde se tiene que colocar o colorear ciertos objetos, respetando ciertas restricciones. Por ejemplo:

- Colocar piezas de ajedrez en un tablero sin que se ataquen.
- Dividir un conjunto de números en subconjuntos con la misma suma.
- Cuadrados mágicos (colocar números enteros en un arreglo/tablero cuadrado de forma que la suma de los números sobre cada fila, cada columna y cada diagonal sea la misma).

Diremos que una configuración es *válida* si satisface las restricciones del problema. El objetivo de un problema sobre configuraciones estáticas suele ser construir/encontrar una configuración válida, encontrar la cantidad de configuraciones válidas o encontrar condiciones para que exista una configuración válida. Por ejemplo, considera el problema:

Ejemplo. En un parque, hay n árboles numerados del 1 al n y n ardillas numeradas del 1 al n , cada una con algunos tejocotes. En el k -ésimo minuto, la ardilla número k va a hacer lo siguiente:

- Elige sus k árboles favoritos y, de entre sus árboles favoritos, elije un sólo árbol para esconder k tejocotes. En los demás $k - 1$ árboles favoritos esconde un tejocote por árbol.

¿De cuántas maneras pueden las n ardillas esconder sus tejocotes si al final todos los árboles tienen la misma cantidad?

En este ejemplo la estructura fija está dada por los n árboles, los objetos a colocar son los tejocotes y la restricción es que al final cada árbol tenga la misma cantidad de tejocotes. En este caso, el objetivo es encontrar el número de configuraciones válidas para este problema.

En algunos casos, un problema puede preguntar sobre condiciones necesarias para que cierta restricción se cumple. Por ejemplo:

Ejemplo. Determina el menor entero positivo n tal que es posible colocar n reinas en un tablero de ajedrez de manera que cada casilla vacía sea atacada por al menos una reina.

Un problema de este tipo (con mínimos/máximos) es en realidad un problema “doble”. Para explicarlo mejor, partiremos de la respuesta que es $n = 5$. Luego, una solución completa consta de dos partes:

- Encontrar/construir una configuración válida con 5 reinas.
- Demostrar que no es posible con 4 reinas o menos.

En estas notas nos enfocaremos solo en la primera parte.

1.2. Configuraciones dinámicas

Por otro lado, en el caso de las configuraciones dinámicas tenemos pasos/movimientos/operaciones permitidos con los que podemos cambiar de una configuración a otra. Por ejemplo:

- Dado un tablero con focos, cambiar el estado (de apagado a encendido o viceversa) de todos los focos en una fila o columna.
- Piezas de ajedrez que se mueven sobre un tablero.
- Borrar un número de una lista y reemplazarlo con otro.

En este caso, se suele partir de una configuración inicial y se busca, mediante los movimientos permitidos, llegar a cierta configuración final. Diremos que una lista de movimientos permitidos es *válida* si nos permite pasar de la configuración inicial a la configuración final.

El objetivo de un problema sobre configuraciones dinámicas suele ser similar al caso estático: construir/encontrar una lista de pasos para llegar a la configuración final, encontrar condiciones para la existencia de esa lista, etc. Por ejemplo, considera el siguiente problema:

Ejemplo. Sea $n \geq 3$ un entero. En un juego hay n cajas en un círculo. Al principio, cada caja contiene un objeto (*pedra, papel o tijeras*), de forma que no hay dos cajas adyacentes con el mismo objeto, y cada objeto aparece en al menos una caja. El juego consiste en mover objetos mediante la siguiente operación:

- Se eligen dos cajas adyacentes y un objeto de cada una de ellas de forma que sean diferentes, y movemos el objeto perdedor a la caja que contiene el objeto ganador.

Por ejemplo, si las cajas A y B son adyacentes y elegimos una piedra de la caja A y unas tijeras de la caja B , entonces movemos las tijeras a la caja A . Demuestra que, aplicando este movimiento suficientes veces, es posible mover todos los objetos a la misma caja.

En este caso, la configuración inicial está dada por el objeto en cada caja en el principio. Existe un único tipo de movimiento permitido con el que podemos mover objetos de una caja para otra. Las configuraciones finales son aquellas donde todos los objetos se encuentren en la misma caja.

Sin embargo, muchos problemas dinámicos en realidad piden determinar si existe una lista válida de movimientos. Estas notas nos ayudarán a resolver el problema cuando la respuesta sea afirmativa. Cuando la respuesta sea negativa, serán necesarias otras técnicas/herramientas.

1.3. Ideas generales

Cada problemas sobre configuraciones es único a su estilo. No hay un método o una idea que funcione en todos los casos. Sin embargo, a

continuación mostraremos algunos consejos e ideas generales que pueden ayudar a resolver un problema sobre configuraciones.

Consejo #0. Familiarizarse con las piezas de ajedrez.

Los tableros de ajedrez constituyen una fuente inagotable de problemas sobre configuraciones. Por lo tanto, puede resultar conveniente estar familiarizado con los movimientos de las piezas de ajedrez y con las regiones del tablero que cada una puede atacar.

Consejo #1. Explorar casos pequeños.

El problema de las tejocotes fue enunciado originalmente con 2026 árboles y 2026 ardillas. Este es un número demasiado grande para explorar cómo se comporta el problema. Sin embargo, en muchos problemas los números son simplemente ser decorativos, pues el problema de los tejocotes puede plantearse con 3 árboles y a 3 ardillas o inclusive puede plantearse el caso 'general (n árboles y n ardillas). Resolver casos pequeños (por ejemplo, los casos $n = 2, 3$) puede dar alguna idea sobre la solución para casos grandes (por ejemplo, el caso $n = 2026$) o para el caso general.

Sin embargo, al momento de elegir casos pequeños debemos ser cuidadosos. Es común que los problemas involucren al número del año, por ejemplo, imaginemos un problema sobre un tablero de 2026×2026 . Podría haber alguna propiedad que haga que el problema no funcione en todos los casos, por ejemplo, una propiedad para tableros de $n \times n$ podría ser válida sólo si n no es múltiplo de 3. Si este fuera el caso, resolver el caso $n = 3$ no sería

útil. Por lo tanto, es importante analizar varios casos.

Consejo #2. Intentar reformular el problema en términos más simples.

En algunas ocasiones, al explorar el problema se puede encontrar información relevante que nos permita transformar el problema en uno más simple. Por ejemplo, considera el siguiente problema:

Ejemplo. Un coleccionista de monedas raras tiene monedas de denominaciones $1, 2, \dots, n$. Desea poner algunas de sus monedas en 5 cajas de manera que se cumplan las siguientes condiciones:

- 1) En cada caja hay a lo más una moneda de cada denominación.
- 2) Todas las cajas tienen el mismo número de monedas y la misma cantidad de dinero (suma de las denominaciones).
- 3) Para cualesquiera dos cajas sucede que entre las dos tienen por lo menos una moneda de cada denominación.
- 4) No existe una denominación tal que todas las cajas tengan una moneda de esa denominación.

¿Para qué valores de n el coleccionista puede hacer lo que se propone?

Por simplicidad, diremos que una k -moneda es una moneda de denominación k . Dado un k fijo, por la última condición, existe (por lo menos) una caja que no tiene k -monedas. Digamos que la caja A no tiene k -monedas. La tercera condición dice que entre cualesquiera dos cajas deben aparecer todas las denominaciones. Así que, las otras cuatro cajas deben tener, cada una, (por lo menos) una k -moneda, pues si otra caja (digamos B) no tiene k -monedas

entonces las cajas A y B no cumplen la condición #3.

Luego, la primera condición dice que no hay más de una moneda de la misma denominación en cada caja. Por lo tanto, para cada denominación, exactamente cuatro cajas tienen exactamente una moneda de esa denominación y la caja restante no tiene monedas de esa denominación. Luego, en lugar de preocuparnos por las denominaciones que hay en cada caja, podemos preguntarnos qué denominaciones faltan.

Dado que todas las cajas tienen el mismo número de monedas y la misma cantidad de monedas, todas las cajas tienen el mismo número de denominaciones faltantes y estas tienen la misma suma. Por lo tanto, existe una forma de guardar las monedas siguiendo las condiciones originales si y sólo si existe una forma de guardar monedas de forma que:

- 1) En cada caja hay a lo más una moneda de cada denominación y cada denominación aparece en alguna caja.
- 2) Todas las cajas tienen el mismo número de monedas y la misma cantidad de dinero (suma de las denominaciones).
- 2) No hay dos cajas que tengan monedas con la misma esa denominación.

En este caso, ya no hay monedas de la misma denominación en cajas diferentes. Por lo tanto, podemos pensar en que estamos separando las n monedas en 5 grupos con la misma cantidad de monedas y con la misma suma. Notemos que esta versión tiene condiciones más simples que el problema original. Además, se puede extraer información valiosa de forma más directa. Por ejemplo, por el hecho de que estamos separando las denominaciones en cinco grupos con la misma cantidad de elementos,

ahora es evidente que n debe ser múltiplo de 5.

Consejo #3. Identificar las partes más restrictivas.

En el problema de los tejocotes, la ardilla número uno coloca solo un tejocote en un árbol, mientras que la ardilla número n coloca n tejocotes en un árbol y en los otros $n - 1$ árboles guarda un tejocote. En este caso, la acción de la última ardilla es mucho más restrictiva que la de la primera ardilla. Por lo tanto, vale la pena observar a las últimas ardillas.

En el problema de las monedas, podemos describir el acomodo de las monedas en dos niveles: primero, veremos cómo se distribuyen las denominaciones y luego nos enfocaremos en las cantidades (de cada denominación y en cada caja). las condiciones #3 y #4 (del enunciado original) imponen las restricciones sobre cómo se distribuyen las denominaciones, mientras que las condiciones #1 y #2 restringen las cantidades. Por este motivo es que comenzamos con #3 y #4.

Consejo #4. Buscar subestructuras.

Los consejos anteriores funcionan para entender el comportamiento del problema y/o reescribir el enunciado. Sin embargo, la parte más compleja de un problema sobre configuraciones es encontrar/construir una configuración válida. Pensar en el problema de forma *global* puede ser muy abrumador, pero en algunos casos podemos pensar en pasos intermedios, es decir, resolveremos solo una “parte” del problema.

Por ejemplo, pensemos en un problema dinámico con un tablero de focos, donde podemos aplicar movimientos para cambiar el estado de algunos focos (de encendido a apagado o viceversa). Una pregunta clásica es determinar si es posible dejar a todos los focos encendidos/apagados.

Como fue mencionado antes, puede ser muy complicado pensar directamente en cómo apagar todos los focos. Una idea que puede ser útil es enfocar nuestra atención en subestructuras. Por ejemplo:

- ¿Es posible cambiar el estado de exactamente un foco?
- ¿Es posible apagar todos los focos de una fila/columna?
- ¿Es posible apagar todos los focos en un subtablero de $k \times k$?

Las configuraciones para resolver estos “subproblemas”, en caso de existir, podrían ser útiles para resolver el caso global.

Consejo #5. Explorar simetrías.

Otra idea útil es proponer/explorar configuraciones con cierto grado de simetría. Por ejemplo, en el problema de las monedas de diferentes denominaciones descubrimos que n debe ser múltiplo de 5. Se puede ver fácilmente que no hay una configuración para $n = 5$ pues deberíamos separar las monedas en cinco grupos de una moneda (cada uno) y con la misma suma, lo cual es claramente imposible. Pensemos en el siguiente caso relevante, $n = 10$. En este caso, debemos separar las monedas en cinco parejas con la misma suma. Una propuesta es la siguiente:

$$\{(1, 10), (2, 9), (3, 8), (4, 7), (5, 6)\}.$$

Notemos que estamos emparejando la menor denominación con la mayor, la segunda menor denominación con la segunda mayor, y así sucesivamente. Además, podemos aplicar esta idea también a las monedas con denominaciones del 11 al 20 para generar cinco parejas con la misma suma. Luego, al juntarlas con las parejas anteriores podemos construir cinco grupos de cuatro monedas con la misma suma:

$$\{(1, 10, 11, 20), (2, 9, 12, 19), (3, 8, 13, 18), (4, 7, 14, 17), (5, 6, 15, 16)\}.$$

Siguiendo esta idea podemos construir cinco grupos con la misma cantidad de monedas y con la misma suma siempre que n sea múltiplo de 10. Notemos que esta construcción se relaciona con el consejo anterior, pues basta con dividir las monedas por bloques de 10.

1.3.1. Ejemplos

A continuación mostraremos las soluciones de los ejemplos anteriores.

El problema de los tejocotes

Primero, calcularemos cuántos tejocotes debe tener cada árbol al final. La ardilla número k esconde k tejocotes en un árbol y en otros $k - 1$ árboles esconde un tejocote. Por lo tanto, en total esa ardilla esconde

$$1 \times k + (k - 1) \times 1 = 2k - 1$$

tejocotes. Así que, la cantidad total de tejocotes escondidos es igual a

$$\sum_{k=1}^n (2k - 1) = n^2.$$

Luego, como todos los árboles tienen la misma cantidad de tejocotes, se sigue que cada árbol debe contener exactamente n tejocotes.

Notemos que, para contar el número de formas en que las ardillas pueden esconder sus tejocotes, no es relevante el orden. En otras palabras, podemos alterar el orden en que las ardillas esconden sus tejocotes sin modificar el problema. Dado que las últimas ardillas esconden más tejocotes, vamos a invertir el orden en que las ardillas actúan, es decir,

- la ardilla número n esconde sus tejocotes en el minuto 1,
- la ardilla número $n - 1$ en el minuto 2, y así sucesivamente.

Notemos que la ardilla número n tiene n formas de elegir al árbol donde esconderá los n tejocotes. Dado que en total hay n árboles, en los otros $n - 1$ árboles esconderá un tejocote. Por lo tanto, después del minuto 1 hay un árbol con n tejocotes y el resto tiene sólo uno.

Denotemos por A_n al árbol donde la ardilla número n escondió los n tejocotes. Dado que en cada árbol se pueden guardar solo n tejocotes, ninguna otra ardilla puede esconder tejocotes en A_n . Por lo tanto, el resto de las ardillas sólo pueden elegir los otros $n - 1$ árboles disponibles. De forma similar, la ardilla número $n - 1$ elige a un árbol A_{n-1} para guardar $n - 1$ tejocotes y en los otros $n - 2$ árboles la ardilla esconde un tejocote (en cada uno). Notemos que ahora los árboles A_n y A_{n-1} tienen n tejocotes escondidos y el resto tienen dos tejocotes (cada uno).

De forma similar, la ardilla número $n - 2$ elige un árbol A_{n-2} en donde guarda $n - 2$ tejocotes y, después de su turno, los árboles A_n , A_{n-1} y A_{n-2} tienen n tejocotes escondidos y el resto tienen tres tejocotes (cada uno). El

proceso continúa así hasta que todas las ardillas terminen de esconder sus tejocotes. Notemos que el número de formas de esconderlos coincide con el número de formas de elegir la lista de árboles $A_n, A_{n-1}, \dots, A_2$ y A_1 .

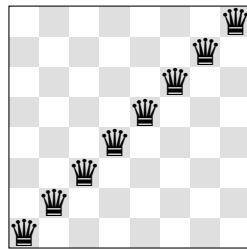
Sin embargo, en cada turno hay un árbol menos disponible para ser elegido. Así que, el árbol A_n puede ser elegido de n formas diferentes, el árbol A_{n-1} de $n - 1$ formas diferentes, y así sucesivamente. En conclusión, la lista de árboles $A_n, A_{n-1}, \dots, A_1$ se puede elegir de

$$n \times (n - 1) \times \dots \times 2 \times 1 = n!$$

formas distintas. Por lo tanto, las n ardillas pueden esconder sus tejocotes en los árboles del parque de $n!$ formas distintas.

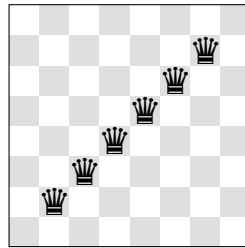
El problema de las reinas

En este tipo de problemas, el primer paso suele ser encontrar una cota, es decir, encontraremos una configuración en donde cada casilla vacía sea atacada por al menos una reina y el número de piezas utilizadas servirá como límite para la solución. Por ejemplo, considera la configuración:



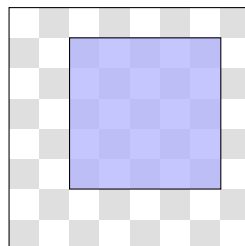
En este caso tenemos ocho reinas en el tablero. Como las reinas pueden atacar a todas las casillas en la misma fila y columna, es claro que cada casilla vacía es atacada por al menos una reina. Así que, la respuesta del

problema es máximo ocho. Sin embargo, podemos modificar ligeramente el ejemplo anterior para reducir el número de reinas.

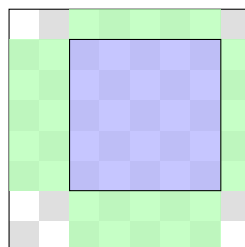


En esta nueva configuración hemos eliminado las dos reinas de las esquinas. Con un poco más de esfuerzo se puede mostrar que cada casilla vacía es atacada por al menos una reina. Por lo tanto, la respuesta del problema es máximo seis. El siguiente paso es determinar si podemos colocar cinco reinas de forma que se cumpla la propiedad.

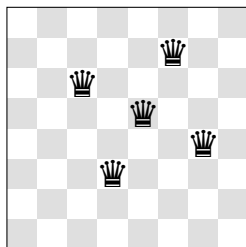
Consideremos un subtablero de 5×5 como se muestra a continuación



Si colocamos una reina en cada columna y fila del subtablero de 5×5 de la figura anterior, entonces la siguiente región es atacada por reinas



Por lo tanto, sólo debemos preocuparnos por cubrir las nueve casillas restantes. En este caso, debemos colocar las reinas dentro del cuadrado azul de forma que cubran (mediante diagonales) las nueve casillas restantes. Después de algunos intentos se puede llegar a la siguiente solución



En este punto debemos preguntarnos si es posible encontrar una configuración válida con cuatro reinas. La respuesta es no, pero como hemos dicho antes, los casos en donde la respuesta sea negativa serán dejados para otro momento. Por lo tanto, el mínimo son cinco reinas.

El problema con piedra, papel y tijeras

Notemos que ciertas configuraciones pueden ser contraproducentes para el objetivo de mover todos los objetos a la misma caja. Por ejemplo, imagina que tenemos tres cajas consecutivas con tijeras, piedra y tijeras. Si movemos las dos tijeras a la caja con la piedra, esa caja se queda aislada pues tendría dos cajas vacías al lado. Esto no significa que sea imposible mover todos los objetos a la misma caja, pero debemos ser cuidadosos en el orden que aplicamos los movimientos pues podríamos quedarnos bloqueados, es decir, llegar a una configuración donde no sea posible alcanzar el objetivo.

Una idea útil es pensar en *(sub)configuraciones deseables*, es decir, en estructuras que podrían simplificar el problema. En este caso, podemos

analizar qué pasa si tres cajas consecutivas tienen tres objetos diferentes.

Supongamos que las cajas A , B y C tienen objetos diferentes (a , b y c , resp.).

Dado que a y b son objetos diferentes tenemos dos casos:

- Caso A: a le gana a b . Luego, b la gana a c , pues deben formar un ciclo. Por lo tanto, podemos mover c a la caja B . En este momento B tiene dos objetos (b y c), pero b pierde contra a y, por lo tanto, b puede moverse a A . Finalmente, el objeto c en la caja B pierde contra el objeto b en la caja A . Así que, c puede ser movido a A .
- Caso B: b le gana a a . Luego, c le gana a b y, de forma similar al caso anterior, los tres objetos pueden ser movidos a la caja C .

Así que, si hay tres cajas consecutivas con diferentes objetos entonces se pueden mover los tres objetos a una misma caja.

El siguiente paso es observar que si hay una caja con los tres objetos, estos se pueden mover a una caja adyacente no vacía. De forma más precisa, supongamos que la caja D tiene los tres objetos diferentes y la caja E tiene el objeto x . Supongamos que los otros objetos son y y z de forma que x le gana a y , y le gana a z y z le gana a x . Dado que y pierde contra x , podemos mover todos los objetos de tipo y a la caja E . Luego, como la caja E tiene un objeto y y z pierde contra y , podemos mover todos los objetos tipo z a la caja E . De forma similar, también podemos mover todos los objetos tipo x a la caja E . Después de estos pasos la caja D queda vacía y la caja E contiene los tres tipos de objetos. Dado que las cajas forman un círculo, podemos repetir este proceso hasta completar todo el ciclo. Al final, tendremos una caja con todos los objetos y el resto estarán vacías.

Notemos que la demostración aún no acaba, pues la conclusión anterior fue obtenida bajo la hipótesis de que existen tres cajas consecutivas con objetos diferentes. Para concluir, es necesario demostrar que esto siempre ocurre. Consideremos tres cajas consecutivas (A , B y C) de forma que la caja A tiene el objeto a y la caja B tiene el objeto b . Supongamos que no hay tres cajas con el mismo objeto, como A y B contienen a los objetos a y b , se sigue que la caja C no puede tener el objeto c . Por otro lado, el problema dice que dos cajas consecutivas no pueden tener el mismo objeto. Por lo tanto, la caja C tampoco puede tener a b y, en consecuencia, la caja C contiene el mismo objeto que la caja A , es decir, C tiene a a .

En resumen, el argumento anterior muestra que si no hay tres cajas con objetos diferentes, entonces A y C tienen el mismo objeto. Si D es la siguiente caja entonces, al repetir el argumento anterior se concluye que B y D tienen el mismo objeto. Por lo tanto, si escribimos a los objetos como lista, obtenemos una lista alternante ($a, b, a, b, \dots$). Sin embargo, esto fuerza a que el objeto c no aparezca en una caja, lo que contradice una de las hipótesis del problema, pues cada objeto debe aparecer al menos en una caja. Así que, siempre hay tres cajas consecutivas con diferentes objetos.

El problema de las monedas

Como mencionamos antes, dado un entero $1 \leq k \leq n$, la condición #4 dice que hay una caja que no tiene monedas de denominación k . Si la caja C_k no tiene monedas de denominación k , las otras tres cajas deben tener monedas de denominación k para satisfacer la condición #3.

La condición #1 indica que cada caja tiene máximo una moneda de cada denominación. Por lo tanto, todo se reduce a ver qué denominaciones están en cada caja o, equivalentemente, ver qué denominaciones faltan en cada caja. La condición #2 dice que todas las cajas tienen la misma cantidad de monedas y, por lo tanto, tienen la misma cantidad de denominaciones faltantes. De forma similar, si cada caja tiene la misma suma entonces la suma de las denominaciones faltantes en cada caja es la misma. Así que, este problema se puede describir mediante las denominaciones faltantes:

- 1) En cada caja hay a lo más una moneda de cada denominación y cada denominación aparece en alguna caja.
- 2) Todas las cajas tienen el mismo número de monedas y la misma cantidad de dinero (suma de las denominaciones).
- 2) No hay dos cajas que tengan monedas con la misma esa denominación.

Por lo tanto, el problema se reduce a dividir las n monedas de diferentes denominaciones en cinco conjuntos con la misma cantidad de monedas y con la misma suma. En particular, es claro que n debe ser múltiplo de 5.

El caso $n = 5$ no tiene configuraciones válidas, pues cada caja tendría una moneda con diferente denominación pero la suma sería diferente. Si tenemos monedas de 10 denominaciones consecutivas, estas se pueden separar en parejas con la misma suma de la siguiente forma:

$$\{(k+1, k+10), (k+2, k+9), (k+3, k+8), (k+4, k+7), (k+5, k+6)\}.$$

Por lo tanto, si n es múltiplo de 10 entonces se pueden separar en cinco grupos con la misma suma (basta con dividir en bloques de 10

denominaciones consecutivas y tomar una pareja de cada bloque).

El caso que resta es el de los impares. Como vimos antes, $n = 5$ no es posible. El siguiente caso para analizar es $n = 15$. Algunas ideas nos pueden ayudar en el proceso de buscar una configuración válida. Por ejemplo, dividamos los números en tres categorías (pequeños, medianos y grandes):

- Denominaciones pequeñas: 1, 2, 3, 4 y 5.
- Denominaciones medianas: 6, 7, 8, 9 y 10.
- Denominaciones grandes: 11, 12, 13, 14 y 15.

En cada caja colocaremos una denominación de cada categoría. Con un poco de esfuerzo se puede encontrar una configuración válida como

$$\{(1, 8, 15), (2, 9, 13), (3, 10, 11), (4, 6, 14), (5, 7, 12)\}.$$

Luego, para impares más grandes podemos aplicar esta configuración para las 15 menores denominaciones y el resto dividir las en bloques de 10. Por ejemplo, para el caso $n = 5 \times 7$ obtenemos la configuración:

- (1, 8, 15, 16, 25, 26, 35).
- (2, 9, 13, 17, 24, 27, 34).
- (3, 10, 11, 18, 23, 28, 33).
- (4, 6, 14, 19, 22, 29, 32).
- (5, 7, 12, 20, 21, 30, 31).

Recordemos que para obtener la respuesta del problema original debemos tomarnos el complemento, pues estamos viendo las denominaciones faltantes. Por ejemplo, para $n = 35$ cada caja debe tener $35 - 7 = 28$ denominaciones diferentes, que se obtienen al tomar los complementos.

1.4. Algoritmos glotones

Un *algoritmo* es una lista de instrucciones precisas que permite realizar una tarea o resolver un problema. Por ejemplo, el siguiente es el algoritmo estándar para determinar si un número n es primo:

- Calcular $\lfloor \sqrt{n} \rfloor$.
- Para cada entero d entre 2 y $\lfloor \sqrt{n} \rfloor$, verificar si $d \mid n$.
- Si alguno divide a n , se concluye que n es compuesto.
- Si ninguno lo divide, se concluye que n es primo.

En muchos problemas sobre configuraciones es prácticamente imposible encontrar/construir una configuración válida de forma explícita, esto puede deberse a la cantidad de pasos necesarios o al tamaño de la estructura. En estos casos, una alternativa es proporcionar un algoritmo que construya una configuración válida. Por ejemplo, considera el problema:

Ejemplo. ¿Es posible elegir 1983 enteros positivos distintos, todos menores que 10^5 , tales que no hay tres números que sean términos consecutivos de una progresión aritmética?

La respuesta es que sí existen esa lista de 1983 números. Sin embargo, como hemos comentado antes, es virtualmente imposible encontrar una lista tan larga con esa propiedad durante el tiempo un examen. Además, para ser una propuesta válida deberíamos verificar que no hay progresiones aritméticas, lo cual requiere miles o hasta millones de comprobaciones. Por lo tanto, vamos a proponer un algoritmo para construir la lista.

Con el fin de simplificar la explicación, diremos que una lista enteros positivos distintos es *espaciada* si no hay tres números que sean términos consecutivos de una progresión aritmética. Por lo tanto, el problema se reduce a determinar si existe una lista espaciada formada por 1983 números menores que 10^5 . En principio, ni siquiera sabemos si existen las listas espaciadas, pero si existe por lo menos una entonces existen infinitas. De hecho, si a cada número de una lista espaciada le sumamos el mismo valor entonces el resultado es una nueva lista espaciada.

Un candidato natural para resolver el problema es encontrar la lista espaciada formada por los números más pequeños. Sin embargo, encontrar tal lista es un problema sumamente complejo. Aunque, en realidad no necesitamos esa lista “minima”, basta con encontrar una lista con números pequeños (sin verificar que sea la lista con los números más pequeños).

Para construir esta lista vamos a proponer un *algoritmo glotón*. Este término proviene principalmente de la informática. La idea principal es tomar la mejor decisión posible en cada momento. Aunque resulta natural elegir la opción más favorable en cada momento, esta estrategia no siempre conduce a una mejor solución. En este caso, el algoritmo glotón que vamos a proponer nos permitirá encontrar una lista espaciada con números relativamente pequeños, pero no es suficiente para afirmar que esa es la lista con los números más pequeños. Aunque, si la lista generada por el algoritmo glotón está formada por números menores que 10^5 entonces esta lista resuelve nuestro problema, no es necesario que sea la “mínima”.

Comenzamos la lista con los números 1 y 2. Luego, a partir del tercer elemento, se selecciona el menor número que no forma una progresión

aritmética con los números anteriores. Por ejemplo, el 3 forma una progresión aritmética con el 1 y el 2. Por lo tanto, el tercer elemento de la lista es 4. Se puede verificar fácilmente que el siguiente número de la lista es el 5. Luego, el quinto elemento no puede ser 6, pues tendríamos la progresión aritmética 2, 4, 6. Tampoco puede ser 7, pues 1, 4, 7 forman una progresión aritmética. Inclusive debemos descartar los números 8 y 9, ya que 2, 5, 8 y 1, 5, 9 son progresiones aritméticas. De hecho, el quinto elemento debe ser el 10. Aplicando el algoritmo, los primeros elementos son

1, 2, 4, 5, 10, 11, 13, 14, 28, 29.

Dado que siempre elegimos el menor número tal que la lista continua siendo espaciada (sin progresiones aritméticas), este es un algoritmo glotón.

El siguiente paso es verificar si el número 1983 de la lista obtenida es menor que 10^5 . Para esto es necesario explorar algunas propiedades de la lista generada por el algoritmo. Por ejemplo, se puede observar que la lista da “saltos”: de 2 a 4, de 5 a 10 y 14 a 28. No entraremos en estos detalles, pero se puede demostrar la lista contiene 2^n elementos menores que

$$\frac{3^n + 1}{2}.$$

Dado que $1983 > 2048 = 2^{11}$, el elemento número 1983 es menor que

$$\frac{3^{11} + 1}{2} = 88574 < 10^5.$$

Así que, existe una lista espaciada con 1983 números menores que 10^5 .

1.5. Juegos

Los *juegos* son un tipo de problemas dinámicos en donde dos o más jugadores realizan movimientos alternadamente y en el que existen condiciones que determinan cuándo un jugador resulta ganador, aunque también puede incluir condiciones de empate. Se usa este término debido a que muchos juegos clásicos entran en esta categoría. Por ejemplo, el ajedrez, las damas e inclusive el juego del gato son (matemáticamente) juegos.

El objetivo suele ser determinar si existe una *estrategia ganadora* para alguno de los jugadores, es decir, si cierto jugador puede ganar independientemente de lo que hagan sus rivales. Al momento de describir una estrategia ganadora siempre debemos pensar que los rivales también quieren ganar y que son suficientemente inteligentes para desarrollar sus propias estrategias.

Un error común es pensar que un jugador debe seguir una estrategia “glotona”, es decir, que siempre va a tomar la opción que parezca mejor en ese momento. Sin embargo, como con los algoritmos glotones, hacer esto no nos garantiza que esa será la mejor estrategia.

Por otro lado, dado que cada uno de los jugadores busca ganar, es natural suponer que si un jugador está cerca de ganar entonces los otros deben intentar impedirlo. Por ejemplo, en el ajedrez, cuando un jugador pone en jaque al rey rival, el adversario está obligado a responder de manera que elimine o bloquee la amenaza sobre su rey.

Un concepto fundamental al momento de describir una estrategia ganadora es la *posición ganadora*. En primer lugar tenemos las posiciones ganadoras

inmediatas, es decir, configuraciones en las que un jugador puede garantizar la victoria en los próximos turnos. Por ejemplo, considera el problema:

Ejemplo. En la pizarra está escrito el número 2. Ana y Bruno juegan alternadamente, comenzando por Ana. Cada uno en su turno sustituye el número escrito por el que se obtiene al aplicar exactamente una de las siguientes operaciones: multiplicarlo por 2, multiplicarlo por 3, o sumarle 1. El primero que obtenga un resultado mayor o igual a 2011 gana. Hallar cuál de los dos tiene una estrategia ganadora y describir dicha estrategia.

Normalmente elegimos un jugador para seguir, en este caso será Ana. Si, antes del turno de Ana, hay un número a mayor o igual a

$$\left\lfloor \frac{2011}{3} \right\rfloor + 1 = 671,$$

entonces en su turno Ana puede obtener un número mayor o igual a 2011 si multiplica a por 3. Por lo tanto, los números 671, 672, ..., 2010 son posiciones ganadoras para Ana. En este caso, nos hemos centrado en la operación de multiplicar por 3 ya que es la que crea números más grandes.

El siguiente paso es preguntarnos, ¿qué pasa con el 670? Si este número está en el pizarrón, entonces Ana sólo puede obtener:

- 2010, al multiplicar por 3.
- 1340, al multiplicar por 2.
- 671, al sumar 1.

Estas eran posiciones ganadoras para Ana, pero si el turno de Ana acaba con estos números entonces Bruno ganará.

De hecho, este es un problema de juegos simétrico, pues cada jugador tiene los mismos movimientos permitidos y la misma condición de victoria. Por lo tanto, si antes del turno de Bruno está alguno de los números 671, ..., 2010, entonces Bruno ganará en ese turno. Regresando al caso del 670, cualquier acción de Ana resulta en una posición ganadora para Bruno. Así que, 670 es una posición *perdedora* para Ana. De hecho, hay dos reglas generales:

- Una posición $\mathcal{P}$ es perdedora si todas las posiciones que se pueden obtener a partir de $\mathcal{P}$ son ganadoras.
- Una posición $\mathcal{G}$ es ganadora si se puede ganar el juego en ese turno o si se puede obtener al menos una posición perdedora a partir de $\mathcal{G}$.

Por ejemplo, dado que a partir de 670 se pueden obtener solo posiciones ganadoras, se sigue que 670 es una posición perdedora. Por otro lado, dado que a partir de 669 se puede obtener 670, que es una posición perdedora, se puede concluir que 669 es una posición ganadora.

Notemos que 668 es perdedora, pues a partir de esta solo podemos obtener 669, 1336 y 2004, que son posiciones ganadoras. Luego, de forma similar, 667 es ganadora pues podemos obtener 668. Siguiendo esta idea, mientras que $2n$ y $3n$ sean mayores o iguales que 671, las posiciones ganadoras y perdedoras se alternan. Notemos que $2n, 3n \geq 671$ si y sólo si $n \geq 336$. Así que, los números impares en el intervalo $[336, 670]$ son posiciones ganadoras y los números pares son posiciones perdedoras.

Todas las posiciones entre 168 y 335 son ganadoras, pues al multiplicarlas por 2 obtenemos un número par en el intervalo $[336, 670]$, y sabemos que esas posiciones son perdedoras. Como con el 670, la posición 167 también es perdedora, pues 168, 334 y 501 son ganadoras.

Con el debido cuidado, esta idea se puede extender hasta determinar si el número 1 es una posición perdedora o ganadora para Ana. De hecho, en este problema el número 1 es una posición perdedora para Ana y, por lo tanto, Bruno tiene una estrategia ganadora.

En este caso, la estrategia ganadora no tiene nada de elegante. La estrategia de Bruno consiste en elegir inteligentemente una operación adecuada para siempre dejar a Ana en una posición perdedora.

En otros problemas sí es posible construir una estrategia ganadora.

Ejemplo. Sea P un polígono regular de $n \geq 5$ lados. Antonio y Benjamín juegan a pintar, alternadamente, los lados y diagonales de P . Antonio usa el color azul y Benjamín el color rojo. El jugador que sea el primero en formar un triángulo con los tres lados de su color es el ganador. Determina todos los enteros positivos n tales que alguno de los jugadores tiene estrategia ganadora.

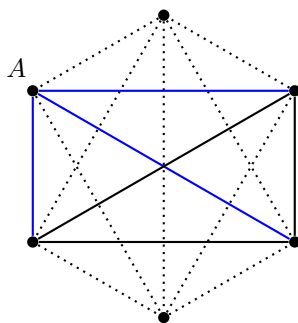
En muchas ocasiones, la pista para resolver un problema puede estar en alguna propiedad o tema conocido. En este caso, Antonio y Benjamín están pintando las diagonales (incluyendo los lados) de dos colores y buscamos triángulos monocromáticos (todos los lados del mismo color). Esto está claramente inspirado en la Teoría de Ramsey.

Con el fin de no extender innecesariamente las notas y evitar alejarnos del tema principal, no explicaré los detalles sobre este tema. Sin embargo, si alguien está interesado puede revisar la siguiente fuente:

- Sección 2.1 del libro *Principio de las Casillas* escrito por José Antonio Gómez, Rogelio Valdez y Rita Vásquez, de la colección Cuadernos de Olimpiadas de Matemáticas (cuaderno color gris claro).

Como recomendación personal, en este nivel no es necesario aprender los teoremas del resto del capítulo 2. Es suficiente con entender la introducción y las propiedades explicadas en la sección 2.1.

Una de las propiedades clásicas de la teoría de Ramsey dice que si pintamos de dos colores todos los lados y las diagonales de un hexágono, entonces siempre se obtendrá un triángulo monocromático, es decir, un triángulo cuyos lados tienen el mismo color. Veamos la siguiente figura:



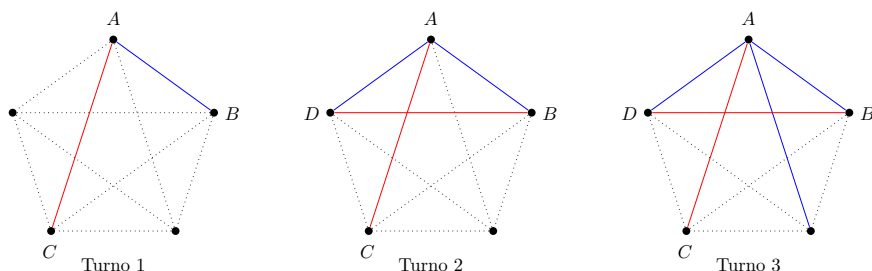
Con el fin de usar el lenguaje usual de la teoría de Ramsey, diremos que un lado o diagonal del polígono es una *arista*. Seleccionemos un vértice A del hexágono. Dado que tenemos dos colores y hay cinco aristas que salen de A , por el principio de las casillas hay por lo menos tres pintadas del mismo color. Supongamos, sin pérdida de generalidad, que desde A salen por lo

menos tres aristas de color azul.

Notemos que con los otros vértices de esas tres aristas se forma un triángulo (negro). Si alguna de las aristas del triángulo negro se pinta de azul, entonces completamos un triángulo azul. Por otro lado, si ninguna de esas aristas se pinta de azul, entonces se pintan de rojo y obtenemos un triángulo rojo. Con base en esta idea, vamos a buscar una configuración con tres aristas del mismo color y con un vértice en común.

Dado que la estructura buscada requiere de solamente tres aristas del mismo color, resulta natural pensar que el primer jugador tiene ventaja para construirla. En su primer turno Antonio pinta cualquier arista ℓ_1 de color azul, y luego Benjamín pinta una de rojo.

Sean A y B los vértices de la arista ℓ_1 . Supongamos que Benjamín pinta una arista que comparte un vértice con ℓ_1 , digamos A . Después del turno 1, hay por lo menos dos vértices que no tienen aristas pintadas. Seleccionamos uno de estos vértices, digamos D , y pintamos la arista ℓ_2 que va de A a D . En este momento la arista que va de B a D no está pintada, y las aristas $\ell_1 = (A, B)$ y $\ell_2 = (A, D)$ son azules. Por lo tanto, para evitar que Antonio gane, Benjamín está obligado a pintar la arista (B, D) de rojo.



Sin embargo, después del tercer turno de Antonio hay dos aristas disponibles para completar el triángulo azul. Notemos que en el turno 3 no se puede completar un triángulo rojo y, dado que se puede pintar solo una arista por turno, sin importar lo que haga Benjamín en su tercer turno este no se puede evitar que Antonio gane en su cuarto turno.

El otro caso, donde Benjamín pinta una arista que no comparte vértices con ℓ_1 , puede ser explicado de forma similar.